

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère deux suites u_n et v_n définies par :

$$\begin{cases} u_0 = 1 \\ v_0 = 2 \end{cases} \quad \text{et pour tout } n \geq 1, \quad \begin{cases} u_n = 2u_{n-1} + \frac{1}{3}v_{n-1} \\ v_n = \frac{1}{2}u_{n-1} + 4v_{n-1}. \end{cases}$$

1. Écrire une fonction Python `suites(N)` qui renvoie une liste contenant tous les tuples (u_n, v_n) , pour tous les $n \leq N$. Testez votre fonction sur $N = 2$ et $N = 4$.
2. Écrire une fonction Python `max_suite_u(N)` qui renvoie le maximum de la liste

$$[u_1 - u_0, u_2 - u_1, \dots, u_N - u_{N-1}].$$

Testez cette fonction sur $N = 5$ et $N = 10$.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes : Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet :

Un relevé topographique est représenté par une **matrice d'entiers**, dans laquelle chaque entier correspond à une altitude mesurée sur une grille régulière. On appelle **pic local** une case de la matrice dont l'altitude est **strictement supérieure** à celle de ses quatre voisins immédiats (haut, bas, gauche et droite).

Remarque : Les éléments situés sur les bords de la matrice ne peuvent pas être des pics locaux, car ils ne possèdent pas quatre voisins.

Exemple :

$$\begin{pmatrix} 10 & 12 & 11 & 9 \\ 14 & 18 & 13 & 8 \\ 9 & 15 & 10 & 7 \\ 6 & 8 & 5 & 4 \end{pmatrix}.$$

Dans cette matrice, la valeur 18 (ligne 1, colonne 1) est un pic local car $18 > 14$, $18 > 12$, $18 > 13$ et $18 > 15$.

Questions :

1. Écrire une fonction `est_pic(matrice, i, j)` qui prend en argument une matrice d'entiers et deux indices i et j , et qui renvoie `True` si l'élément situé à la ligne i et à la colonne j est un pic local, et `False` sinon.
2. Écrire une fonction `compter_pics(matrice)` qui prend en argument une matrice d'entiers représentant le relevé topographique et qui renvoie le nombre total de pics locaux présents dans cette matrice.

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: Nous allons écrire un programme permettant de faire le suivi des pas journaliers. On dispose d'une liste `pas` d'entiers, où `pas[i]` est le nombre de pas effectués le jour `i`.

Exemple :

```
pas = [3200, 5100, 4900, 8000, 8000, 2000, 6500]
```

1. Écrire une fonction `jours_actifs(pas, seuil)` qui renvoie la liste des indices des jours où `pas[i] >= seuil`.

Exemple : `jours_actifs(pas, 6000)` renvoie `[3, 4, 6]`.

2. Écrire une fonction `plus_longue_serie(pas, seuil)` qui renvoie la longueur de la plus longue suite d'indices **consécutifs** dans la liste calculée par la fonction de la question 1.

Exemple : ici `J = [3,4,6]`, donc la plus longue série vaut 2 (3 puis 4).

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: Durant un cours de sport, un professeur souhaite organiser un tournoi de tennis de table. Sa classe comporte 26 élèves, qu'il numérote de 1 à 26. Il dispose de 13 tables sur lesquels les élèves s'affrontent. On représente les terrains par une liste de tables, et chaque table par un tuple constituant une paire d'élèves. Par exemple, une disposition de départ des élèves est la suivante:

$[(1, 2), (3, 4), (5, 6), \dots, (23, 24), (25, 26)]$

1. Une fois un match terminé sur une table, on souhaite que le premier élèves du tuples correspondant soit le vainqueur. Écrire une fonction Python `match` qui prend en entrée un tuple d'élèves, et choisit un gagnant au hasard et renvoie un tuple adapté au résultat du match.
2. Lorsqu'un élève gagne, il monte à la table voisine d'indice supérieur. Lorsqu'un élève perd, il descend à la table voisine d'indice inférieur. Le perdant de la table d'indice 0 ne change pas de table, et il en va de même pour le gagnant de la table d'indice 12. Écrire une fonction Python `round` qui prend en entrée une liste de tuples représentant les tables, et qui simule un match sur chaque table et renvoie la liste des tables après que les élèves se soient déplacés.

Pour générer aléatoirement des nombres entiers, nous rappelons qu'on peut utiliser la fonction `random.randint(a,b)` qui renvoie un nombre entier entre a et b .

Exemple d'usage:

```
import random
print(random.randint(2,15))
```

Remarques étudiant :
Remarques correcteur :

IN100 Initiation au Python

Contrôle continu 2

Étudiant :	Heure Début	Note (/5)
Correcteur :	Heure Fin	

Consignes: Vous avez 20 minutes pour coder la solution. Seules les types et les instructions vues en cours sont autorisées. Tout usage d'une fonction venant d'une bibliothèque qui n'a pas été vue en cours ne sera pris en compte.

Sujet: On considère un flux de messages représenté par une liste de chaînes de caractères. Chaque message est associé à un niveau de priorité : un message *urgent* commence par le caractère `!`, tandis qu'un message *non urgent* ne commence pas par `!`.

Le but est de séparer ces messages en deux **files**, une pour les messages urgents et l'autre pour les messages non-urgents. Pour cela, on utilise :

- une liste **messages** représentant une file de message, dans l'ordre de leur arrivée.
 - deux **files**, une pour les messages urgents et l'autre pour les messages non-urgents.
1. Écrire une fonction Python **enfiler(f, x)** qui ajoute le message **x** dans la file **f** et une fonction **defiler(f)** qui enlève un élément de la file **f**.
 2. Écrire une fonction Python **traiter_messages(messages)** qui traite les messages de la file **messages** un par un et :
 - ajoute le message (sans le caractère `!`) dans la file **urgents** s'il est urgent,
 - ajoute le message dans la file **non_urgents** sinon.

Vérifier que tous les messages ont été traités et que la file **messages** est vide à la fin du processus.

Remarques étudiant :
Remarques correcteur :